

Beyond Requirements Dictator

How Agile Helped a Business Analyst Discover Her Real Value

Diane Zajac-Woodie
Business Analyst/Agent of Change
Erie Insurance Group
Erie, Pennsylvania, United States
diane.zajacwoodie@gmail.com

Abstract—This is my personal journey of discovering how to provide value as a business analyst on an agile team. I will share some things that smoothed my path and some obstacles that slowed me down.

And for the many BAs struggling with how they fit into this new world, I will also redefine the role of a business analyst as more than a requirements dictator. In closing, I will illustrate how BAs, as well as the rest of the team, can reap the benefits of broadening their roles and stepping outside of their role boundaries.

Keywords—component; software development; agile; business analysis; teams; collaboration; crossing role boundaries; continuous learning; value; coaching; obstacles; testing; facilitation; requirements; documentation; culture change

I. INTRODUCTION

I spent many years blissfully unaware of technology except when I needed to reboot my pc. Then, at a time when I was looking for a career change, I stumbled onto a job description for a “Business Analyst.” I had never heard of this position, but I was intrigued enough to apply.

When I started as a BA, I would obsess over every task that I learned. I submitted in depth requirement plans, created detailed meeting minutes, and precisely documented complex requirements. I colored within the lines. I was new to IT and I thought these tasks were a reflection of my value to the organization.

After a few years, I was placed on one of the few agile teams at my large company. Our first agile coach started challenging our team’s assumptions and teaching us a different approach to our work. It was exciting to me, but equally unsettling. I began to question my own worth. If no one needed my checklists and documentation, then what was I supposed to do?

II. THE PATH

I fretted over this question. On the one hand, I was enthusiastic about getting rid of excessive process. Requirements changed frequently and I knew that no one read the documentation. It was clearly wasteful to me. But on the

other hand, the traditional BA tasks that I recently learned seemed to disappear on an agile team.

Our coach was instrumental in helping me realize that I had a lot more to offer. He suggested that:

Our value is not in the tasks that we do.

Hmmm. Up until that point, my self-worth was wrapped up completely in my tasks. I had lots of ideas, but pushed them aside in favor of a checklist of deliverables that someone else deemed important.

To be honest, when he first told me that I should write “tokens of conversations” on index cards, I thought he was a little crazy. My contribution to a project had always been creating detailed spreadsheets. He wanted me to make a few notes on the back of an index card? I was eager for a better way of working, but I was more than a little skeptical.

Through many, sometimes heated, discussions, he showed me that our team could produce more valuable software if we communicated and collaborated. He also taught us core practices like pair programming and acceptance test driven development. But more than being knowledgeable, he was patient, supportive and encouraged challenging questions. He taught everything he knew to anyone who wanted to learn.

Several people on the initial team were vocal about not wanting to learn this open, iterative way of working. The team had been assembled based on project needs and staff availability. Let’s just say that no one was given a choice. Eventually management realized the importance of having willing participants and the resistors were transferred off the team. This was essential to creating a more open, collaborative environment.

I also personally benefited from attending agile conferences and joining local agile groups. It was encouraging to meet people who survived their own organizational transformations! And once our coach left, I could reach out to a supportive community of practitioners for advice or just a healthy debate on a topic. My knowledge and confidence grew in large part to these interactions.

III. THE OBSTACLES

There are many things that would have smoothed my path to agility. The first step would have been formal training earlier

in the journey. One of the developers on our team had previous agile experience. Initially, the expectation was for him to teach the rest of us as we worked on delivering projects. He was very knowledgeable, but could not teach us the basics while addressing our unique questions and concerns. In addition, the pressure to deliver was too great.

Even once we hired a full-time coach, some team members were reluctant to embrace working in this type of environment. As I mentioned before, no one was given a choice. This caused us to spend a lot of time debating every suggestion for change. Getting team consensus to try something was slow and tiring. Thankfully, as our team membership changed, the team's progress improved.

Another challenge for me personally was the lack of information available for non-development roles on agile teams. I would search for websites and books to gain understanding of my new responsibilities. Instead I found endless information around coding practices and QA methods. Most references to business analysis stated that a BA on a newly formed agile team should choose between the scrum master or product owner roles. I persevered.

But perhaps the most frustrating challenge was coping with the corporate culture that surrounded me. Like most large organizations, my company encourages and rewards individual efforts. And when people view themselves as individual contributors with individual goals, it is difficult to rally around a shared team vision. This solo mindset further encourages silos within the team.

Another factor that adversely affected us was that several roles were matrixed to our team. That meant, for example, that our QA team member reported to the QA department rather than to the rest of the team's Software Delivery department (see Fig. 1). This created a subtle sense of separation as we attended different section meetings and Christmas parties.

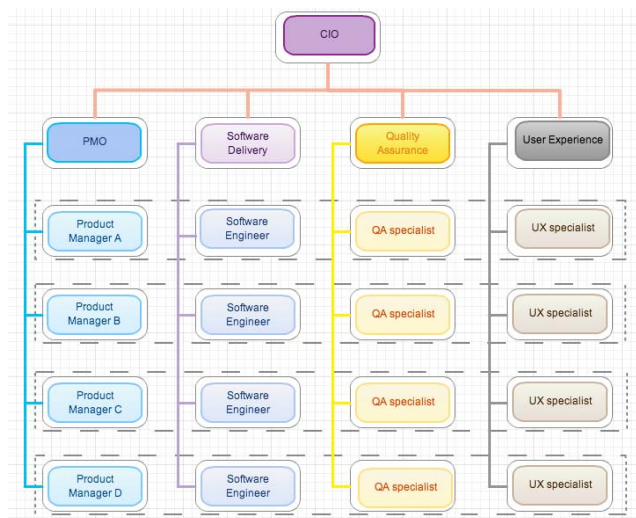


Fig. 1. Sample of a matrix organization chart.

More troubling was that these team members were splitting their time and energy amongst multiple projects. To complicate matters, the projects external to our team were using waterfall methodologies. So our “standing team” was learning to focus on delivering the highest business value for our business partners in the leanest way possible. And these matrixed individuals were struggling to keep up given all the context switching forced upon them.

A lot of these challenges were the direct result of intelligent, well-meaning individuals. This underlines the immense impact that culture plays on adopting an agile mindset. I faced many difficult and disappointing conversations with co-workers who were just not ready to embrace a different way of working.

I know that changing the culture at a large, conservative insurance company will be a slow, sometimes frustrating, process. But I also know that culture, by its very nature, continues to evolve. I take solace in knowing that I can help to influence the direction that it goes by continuing to educate and challenge the people around me.

IV. WHAT STAYS THE SAME

So despite the obstacles, I realized that my fears of becoming obsolete were unwarranted. The role of a business analyst can certainly survive and even thrive in an iterative environment. “What” a BA provides will stay somewhat the same, but “when” and “how” things are accomplished will change.

The most common description of the BA role is as a facilitator between the business and IT. We help to ensure quality communication between these, sometimes opposing, groups. These facilitation skills can be even more useful during and after an agile transformation.

In most companies, it is the IT side that introduces agile concepts. Our business partners can sometimes be hesitant since they don't know what it means to “go agile.” The established relationships that a BA has with the business can help alleviate any apprehension outside of IT. And a trusted BA is in a unique position to highlight the benefits for the entire organization.

In addition, I am still charged with eliciting requirements. I just no longer produce a large, detailed document at the beginning of a project. I now guide the team to progressively elaborate features. At the very start of a project we may have a dozen high-level stories to help define scope. As we dive into the work, we break those down further to capture the details, focusing on small pieces of demonstrable software.

While details are essential, creating an overall vision of a project is equally important. One practice that I continue is helping to develop business process models (BPM). We have a group that specializes in BPMs so I collaborate with them to create the workflows for any new functionality that we are planning. We share these with stakeholders, with our team and with anyone who is contributing to the project (see Fig. 2).



Fig. 2. Team members discussing a Business Process Model for a complex workflow.

For a complex workflow with a lot of exceptions, they may contain a lot of detail. Despite this, we try to keep the models as flexible as possible. When new information is discovered or a change is needed, we simply put post-its on the model itself. We keep them hanging near our team space so we can refer to them as we discuss stories. When enough changes are needed, our partners update them to a new version and re-print them.

It is a given that the BA is charged with reaching out to various people, from stakeholders to SMEs, from developers to quality assurance. In my pre-agile days, I remember having 3-5 meetings a day with different groups trying to gain and share understanding in 60-minute windows of time. I took this knowledge back to my functional silo, where I would create a requirements document in isolation.

I still regularly communicate with anyone and everyone, but with a minimum need for formal meetings. I invite stakeholders to our team space where we have focused conversations on their specific expectations for an upcoming feature. I work with our product owner to ensure our story cards stay prioritized. I spell out acceptance criteria with our QA teammate. These are just a few examples of focused collaboration which is more engaging and therefore, more productive.

Documentation of these conversations is still important. If there is a need to record some of these interactions, I try to keep it as lightweight as possible. For example, I still articulate an agenda to the person or group, because defining the purpose of meeting helps to provide focus. But I may just write our problem statement on a whiteboard and capture the conversation below as it unfolds.

And I still produce and share meeting summaries for important decisions. This may include a snapshot of the whiteboard and a few sentences for clarification. The key is

that I do not produce documentation without questioning what value it adds.

V. WHAT CHANGES

There are a lot of BA tasks that appear to disappear on an agile team. And some of those have disappeared for good reason because they were not adding real value. In reality, the important tasks have evolved so even better outcomes are achieved.

In the past, when there were multiple alternatives to solve a problem, I would generate documentation to describe the pros and cons of each. Now I partner with user interaction specialists to map out and prototype solutions that we can discuss and tweak with our product owners. I have found that comparing mock-ups is a much more effective means for sharing options.

I am also a huge proponent of story mapping - a technique made popular by Jeff Patton that helps us discover, visualize, and plan desired features [1]. In the past, our project teams might have read a project initiation document when they were first assigned to a project. From this, they gained a high level understanding of the goals. Then they went back to their current project while they waited for requirements.

Now at the start of a project, I invite all the stakeholders to participate, along with the development team, in a story mapping session. The group discusses the business goals and the possible user activities and tasks needed to achieve it. This generates a common understanding of what the project hopes to deliver.

Building this initial story map really opens the dialogue between all parties involved. In the past, the development team may not have ever spoken to any stakeholders. It was generally expected that, as the BA, I was the go-between. Before story maps, I did not have a lightweight, visual technique that fostered such collaborative communication.

Because of its simplicity, a story map is easy to keep current. Once it is created, I transfer the map to Excel, with each cell representing a post-it (see Fig. 3). This makes it easier to share and easier to insert tasks or stories if we discover that we missed something¹. We typically reference it on the shared big-screen television in our common workspace. And if needed, I can print it to take to a meeting.

¹ There is a promising new tool called CardBoard [2] that allows for story map creation along with saving release "slices." I was a beta-tester and I would highly recommend checking it out. I have no financial or legal connection with the product.

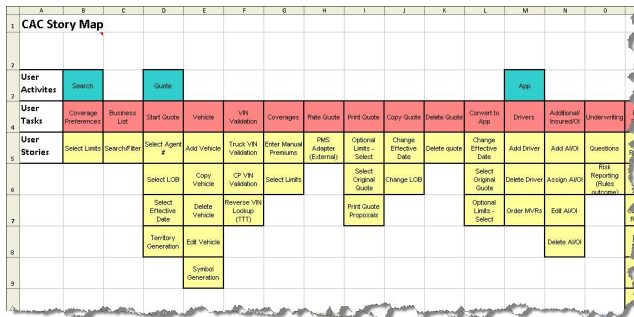


Fig. 3. Sample of a story map saved in Excel.

One of my favorite differences in this new environment is around testing. I remember creating requirements that were “testable,” that our quality assurance engineers could trace to their test scripts. Now I work side-by-side with QA to write executable test scenarios. Using Behavior-Driven Development (BDD), we spell out requirements in a ubiquitous language that literally becomes our automated tests.

More important than how any specific task has changed is the change in timing of the tasks. The requirements phase is at the front end of a traditional project. While the BA typically stays involved throughout the lifecycle, the majority of their effort is spent up front, eliciting, gathering and documenting.

This is based on the myth that requirements will not change so you can capture them all at the start if you ask the right questions. Asking good questions is still an essential skill for a BA. But on an agile team, the BA, along with the team, focuses on one story at a time. This allows for “just-in-time” elaboration of the details.

When requirements are uncovered throughout development, they do not have the chance to become stale. And the team does not waste time refreshing their memories about decisions that occurred months ago. There is an ongoing dialogue with stakeholders so information is always current.

VI. BA SKILLS EVEN MORE VALUABLE

So what about BA skills other than requirements elicitation, facilitation and documentation? Is there a place for creativity, consensus-building and problem solving skills on an agile team? I firmly believe that these skills and many other BA assets are even more valuable.

In a highly collaborative environment, being able to steer a conversation toward an agreement is a vital skill to possess. “Agile” or not, when you get a group of people with different ideas together, it helps to have a referee in one form or another. They make sure all perspectives are heard and that realistic solutions are actually discovered.

Another aspect of business analysis that thrives in this environment is getting to the root cause of a problem. Our goal is to develop the simplest thing that will solve a business problem. So I help to prioritize our work in order of most important to least. As we develop and demonstrate the working software, I challenge the project sponsors to consider whether

the intent of the project has been fulfilled. Do we really need the remaining stories to solve the original problem?

This has been met with mixed results. One of the biggest surprises for me has been the resistance on the business side to embrace an agile philosophy. Because the entire project is approved at the onset, the mindset seems to be, “We want it all.” Even with open dialogue, it can be difficult to convince stakeholders to break up bigger features and deliver less than originally scoped.

This requires teaching the stakeholders how to think differently. They can’t imagine the value of X without Y and Z. My job is to educate them about the benefits of delivering the most valuable item, X, as quickly as possible. Then we can gain feedback that can inform whether they even need Y or Z. The project sponsors may agree in theory, but struggle with breaking free from the prevailing mindset of an “all or nothing” release.

On a positive note, with its transparency and flexibility, agile relieves the BA of being the “scope police.” The product owner polices his or her own scope decisions, so instead of saying “no,” I have become a sort of accountability partner. I show them options for slicing features and help them to see the trade-offs of business value versus development effort. Creativity and good communication skills have been essential for this shift.

An essential part of my partnership is openness and honesty. As a BA, I am in a perfect position to glean and share information because I work with all the roles on the team in some form or another. The project manager might get the status of everyone’s work, but the BA actually works with everyone. From PM to QA to developer, the BA understands the specific challenges the team is facing.

Because of this central role, the BA is usually a key driver to creating visual tools, or information radiators. Also called “Big Visible Charts,” these can include anything the team deems important enough to track and share, such as number of tests written, code coverage, or build status [3].

One of the common types of information radiators is a burn chart². I work with our project manager to create burnup charts that reflect changes in scope and velocity (see Fig. 4). As the BA, I also shepherd the story card writing so that our work-in-progress is transparent. On my last team, I worked with developers to configure a tool so that the BAs and QA could run the automated test suite on demand and see results for failing tests.

² I prefer burn up charts because they reflect changes to scope, along with velocity. Burn down charts simply show how much work is left to complete. They can obfuscate changes to scope because work is being completed at the same time that scope changes.

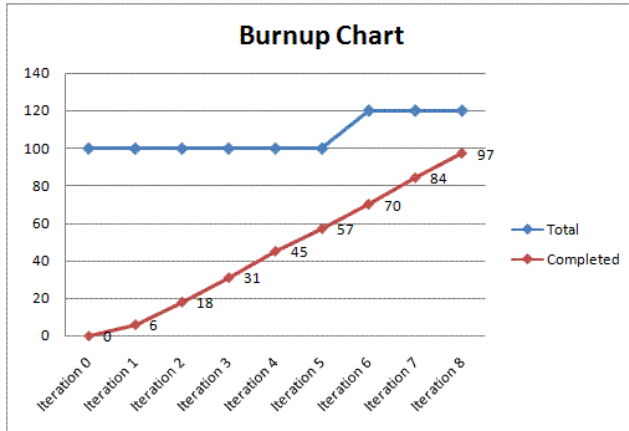


Fig. 4. Sample burn up chart.

In addition to information radiators, everyone on the team should care about retrospectives. But a lot of teams get caught up in delivery work and need someone to remind them to inspect and adapt. I am the biggest proponent of regularly focusing the team to identify what is working and where we can improve. A BAs continuous improvement mentality and facilitation skills are very useful to encourage active participation from all team members.

VII. ERASING ROLE BOUNDARIES

Most business analysts have found themselves crossing role-lines in some form, from helping business sponsors write business cases to performing some functional testing at the end of a project. In fact, most people who care about the success of a project will lend a hand when needed to get a project across the finish line.

This attitude should define a high performing team. When the team has a shared vision of success and a narrow slice of work to focus on, role boundaries disappear. The BA can write test scenarios and a developer can ask the product owner what an error message should say. All that matters is keeping work, which is ultimately business value, flowing across the board.

This usually happens over time if given the right conditions. My first team started by sitting together and focusing on smaller pieces of work. In effect, we shrank the development cycle to the time it took to complete one card. We used to have hour-long meetings to officially hand-off work between phases, like requirements to design or development to testing. These were replaced with frequent ten-minute conversations throughout the iteration.

To further encourage teammates to cross roles, we implemented work-in-progress (WIP) limits. On our storyboard, each activity related to delivery, such as analysis, development or testing, had a story card limit. Everyone agreed to the limits, which were based on the team's size. The idea was that if any WIP limit was reached, then the team would have to help with that activity in order to move a story.

For example, if the testing WIP limit was one and one story was undergoing testing, then the WIP limit for testing was reached. If the developers completed two more stories which were then ready to be tested, then the developers had to help test. They could not just move the stories to testing and start development on more stories.

When our team respected WIP limits, it focused the whole team on getting a few stories to 'Done' at a time. This focus on completion caused team members to frequently step into other roles to remove roadblocks. To be successful, it was important to have a team agreement that the stories closest to 'Done' were the highest business value and therefore the highest priority.

Even so, we are human and it was tempting to pick up work that we felt was interesting or easy or without obstacles. Not many people want to hunt down answers or come up with solutions to roadblocks. This is especially true if there is other work available. So it was important for us to hold each other accountable with friendly reminders to stay focused on the highest priority work.

At the onset of our team formation, most members stayed in their safe zone, their specialty. I did the same. So much was changing around me that I wanted to hold onto at least one thing that was comfortable. Then I started helping our PM with burn charts and release planning. It was a small step outside my realm of expertise. As I learned more, I appreciated being able to contribute to these conversations.

Soon the whole team was discussing testing strategies and automated approaches. While I could not contribute technical details, I gained an understanding about our testing goals and our plan to get there. Then, as I mentioned previously, I started pairing with our tester and developers to write executable requirements (see Fig. 5).

```

Feature: Log in to Online Account
  - As a customer
  - I want to log in to my online account
  - So that I can manage various account functionality

Scenario: Successful log in with valid username and password
  Given I have an active Online Account
  When I log in to my account
  Then I should see My Account Home

Scenario: Display an error message when username field is blank
  When I log in to my account without a username
  Then I should see the required username error message

Scenario: Display an error message when password field is blank
  When I log in to my account without a password
  Then I should see the required password error message
  
```

Fig. 5. Sample of executable test scenario.

Writing requirements as testing specifications has been one of the biggest adjustments for our team. It required shifting our mindset from "I know everything that I need." to "What can I learn from you?" We needed to embrace that each person brings a different and valuable perspective.

This mindset affected everyone on the team. It opened up learning opportunities for each person. Our project manager paired with the developers to help talk through technical solutions from a different angle. The developers worked through scenarios with our business sponsors to gain insight. Our tester and I even learned a little Ruby so we could tweak our regression tests if needed.

With each collaboration, each person benefited by expanding their knowledge. Those learning new skills were stretched beyond their area of expertise. And those doing the teaching were given the opportunity to deepen their understanding since a proven way to master a skill is to teach it to someone else. And perhaps the biggest benefit is that the resulting solutions were better than what they would have been if we had worked solo.

VIII. THE JOURNEY CONTINUES

As I ask people about their experiences with business analysts, I find that there is no such thing as a typical BA. That is because one of our strengths is to adjust our actions based on the context of a given project. The BA helps define the business problem and guides the team to the best solution. And these problems and solutions are different every time.

Even so, I have learned that my real value does not come from my actions. My real value comes from bringing the best out of every person with which I work. Sometimes this is achieved by pushing a person beyond their assumptions.

Sometimes it involves encouraging lively debates. Other times it requires listening beyond the words being spoken.

This journey of discovering my real value has been one of continuous questioning, continuous sharing, continuous adjustment. I have learned from the frustrations as much as from the celebrations. With an attitude of lifelong learning, I plan on continuing to push boundaries and to stretch my knowledge. This continual metamorphosis is what “being agile” has meant to me.

ACKNOWLEDGMENT

Endless thanks goes to my first coach, Michael “Doc” Norton, for opening the door for me and for his continued friendship. I am also grateful to Matt Barcomb for seeing more in me than I could see in myself. In addition, I would like to thank Kathy Grignol and Dave Soule for giving me space to grow and explore. I might have given up without it.

Many people have influenced, supported and put up with me along my journey. And no one has put up with more than my husband, Shawn. Thank you for your unending patience and encouragement. ☺

REFERENCES

- [1] http://www.agileproductdesign.com/blog/the_new_backlog.html
- [2] <https://www.cardboardit.com/>
- [3] <http://alistair.cockburn.us/Information+radiator>